

Azure:

Database Elevate Privilege & Remote Code Execution & Effect other User's Metric

Platform:

Azure Database for PostgreSQL Flexible Server

Class:

Remote Code Execution

Summary:

I found a new database elevate privilege problem in PostgreSQL Flexible Server which can make me to be superuser in database. With superuser's role and a new code execution technology, I still can get a code execution over PostgreSQL container. Based on vulnerability [VULN-125653](#) which I have reported, find that I can still get code execution in VmAgent.SideCar container (for better understanding, I named PostgreSQL container and C# container). Reverse VmAgent.SideCar.Postgres.dll with ILSpy, I find that it has some change in MDMMetricsSender class, but the impact is minimal, I can manual generation udp packet about metric on the full knowledge of MDMMetricsSender. And mdmd.p12 is still in the "/tmp" directory which the permission of it is readable by other users, I get a private key and public key without password after I download and parse it, just create MDM environment from image linuxgeneva-microsoft.azurecr.io/genevamdmd:latest, then put some specially modified metric udp packet to local mdm server, **I find that I can still control any other user's metric info.**

Note: It should have the knowledge about subscription id and resource group name to change any other user's metric.

Attack Description:

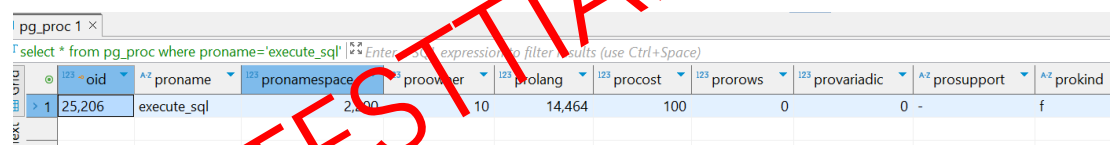
Phase I: Get superuser's role of database

The new vulnerabilities is existed in extension 'anon', which can be used to get superuser's role (azuresu).

- **There is still the problem in extension Anon which I mentioned in [VULN-107034](#) and [VULN-125653](#) before, but a variant.**
 1. Create a flexible server postgres database with extensions anon and shared_library anon enabled. Connect in database postgres.
 2. Create a new schema test, set anon's maskschema='anon' and sourceschema='public'.
 3. Create table public.pg_masking_rules with the same attribute as anon.pg_masking_rules. Find that you can control view anon.pg_masking_rules.

4. Create function public.execute_sql with security definer and set sourceschema='test', create a table test in schema test and get its attrelid.
5. Insert a malformed SQL content with attrelid to public.pg_masking_rules, then select anon.mask_update(). (The SQL content is used to alter public.execute_sql owner to azuresu)
6. Check whether the owner of the function has been changed to 'azuresu'. And Get superuser role running state but can't create or alter superuser. A concise illustration is shown in Figure 1.

The root cause of this problem is that anon's superuser role and "security definer" in function mask_update.



oid	proname	pronamespace	proowner	prolang	procost	prorows	provolatile	prosupport	prokind
25206	execute_sql	2	10	14	100	0	0	f	f

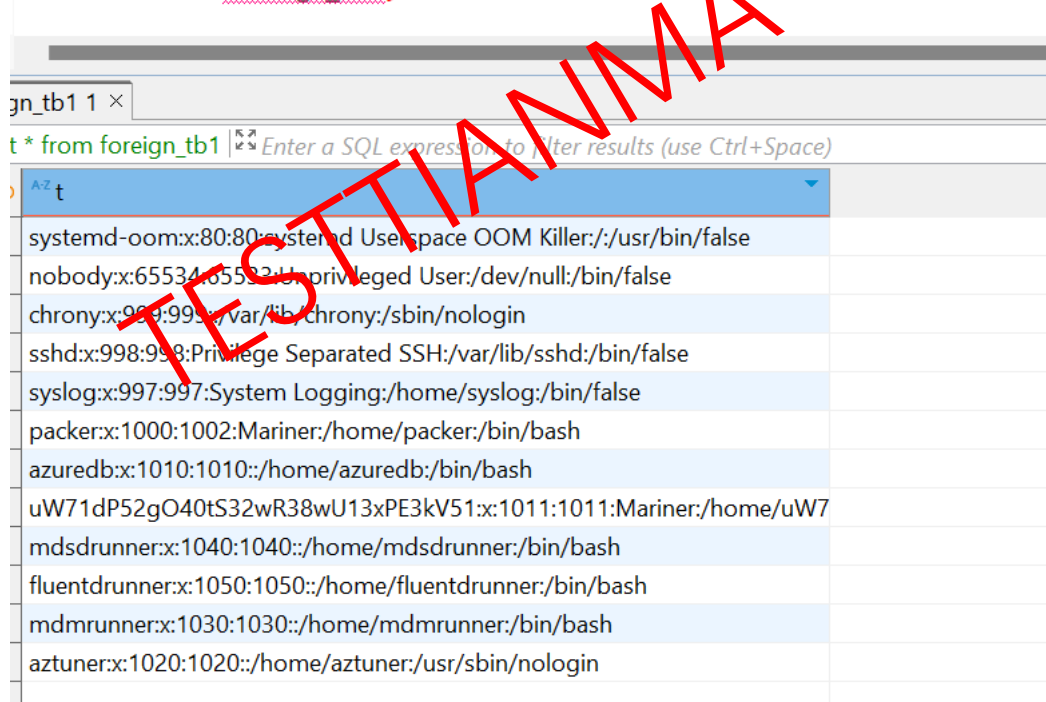
Figure 1 The "security definer" function execute_sql with owner azuresu

Phase II: Code execution over PostgreSQL container

There are many ways to get code execution with PostgreSQL, in this time I use extension file_fdw as first-time command execution to suite an environment that indirectly utilizes superuser privileges. After that I choose UDF to realize it. You can find fully using in https://github.com/testtianmaaaa/postgresql_udf_help. And a briefly look in Figure 2.

1. Create extension file_fdw in function execute_sql.
2. Create foreign table file_fdw with program and grant the privilege to user.
3. Get code execution like Figure 2, then use UDF with azcopy command and token in "datadrive/SasTokenForLogArchiveTarget.txt".

```
select * from foreign_tb1;
```



t
systemd-oom:x:80:80:systemd User:space OOM Killer:/usr/bin/false
nobody:x:65534:65534:Unprivileged User:/dev/null:/bin/false
chrony:x:999:999:/var/lib/chrony:/sbin/nologin
sshd:x:998:998:Privilege Separated SSH:/var/lib/ssh:/bin/false
syslog:x:997:997:System Logging:/home/syslog:/bin/false
packer:x:1000:1002:Mariner:/home/packer:/bin/bash
azuredb:x:1010:1010:/home/azuredb:/bin/bash
uW71dP52gO40tS32wR38wU13xPE3kV51:x:1011:1011:Mariner:/home/uW7
mdsdrunner:x:1040:1040:/home/mdsdrunner:/bin/bash
fluentdrunner:x:1050:1050:/home/fluentdrunner:/bin/bash
mdmrunner:x:1030:1030:/home/mdmrunner:/bin/bash
aztuner:x:1020:1020:/home/aztuner:/usr/sbin/nologin

Figure 2 code execution example

Phase III: Change other user's metric

I test the previously discovered vmagentsidecar command injection vulnerability again. It is still existed, which I got the new version about VmAgent.SideCar.Postgres.dll showed in Figure 3.



Figure 3 A little Changes in Class MDMMetricsSender

After that, I find mdmd.p12 in the /tmp directory again. Just download and parse it which you can see in Figure 4. Because of VULN-150037, I know that Azure has endpoints for metric submission like <https://azossdb1-black.red.prod.microsoftmetrics.com/api/v1/data/metrics/mdm-stream/monitoringAccount/xxx?replicate=false>.

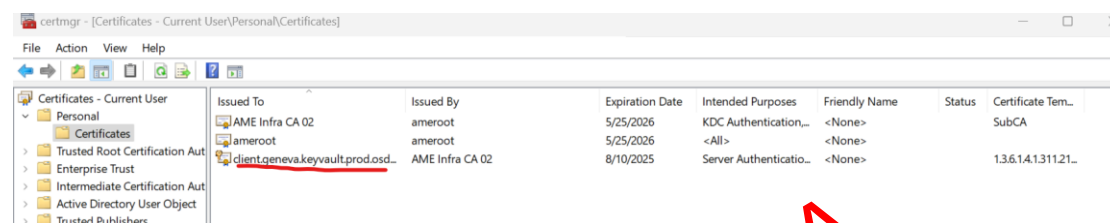


Figure 4 Cert details

I use this cert to access these URL successfully. If I don't supply the cert, it will prohibit me from accessing. I then try region-level switching like canadacentral to eastasia which is also successfully accessed. But failed in "MicrosoftDBForMySQLFlexShoeboxIrodeastasia", the results is presented in Figure 6.

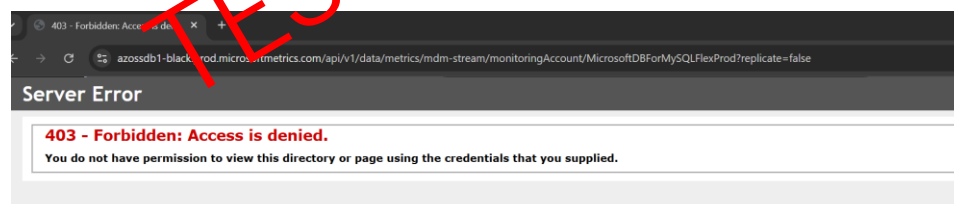


Figure 5 Metric endpoint URL without cert



Figure 6 Metric endpoint URL tests

Usage:

I will give the poc which name poc.sql I have used in attachment. Please pay attention to my comments. The PostgreSQL udf is still in https://github.com/testtianmaaaa/postgresql_udf_help.

And I use the Blob's sas token in directory '/datadrive' to transmit files I needed like Figure 7.

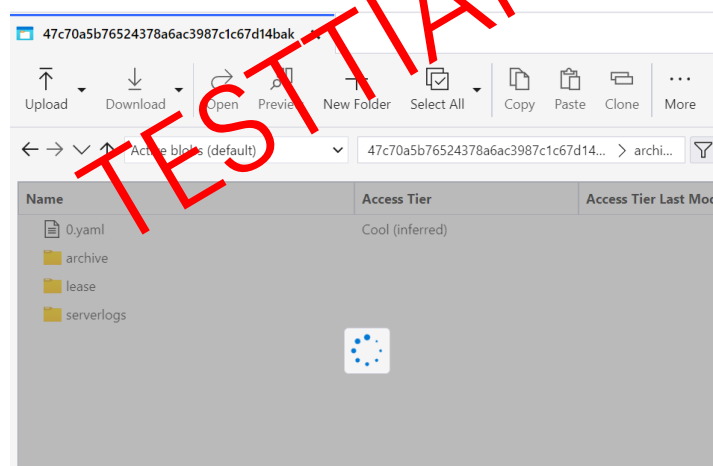


Figure 7 Azure storage list